

Model card — escalation-risk prompt

Version: schema 1, trained 2026-09-11 **Artefact:** `backend/app/ml/risk_model.json` **Training code:** `backend/scripts/train_risk_model.py` **Metrics:** `backend/eval/results/risk_model.json`

The one thing to read first

This model is trained on synthetic data and has never been validated against real clinical outcomes. The numbers below describe its behaviour on a held-out slice of a cohort generated by a documented rule in `backend/app/ml/synthetic.py`. They demonstrate that the machine-learning pipeline is correct — features are extracted as intended, the model recovers a signal it cannot see directly, calibration does what calibration is for, and the code that serves predictions agrees with the code that trained them.

They say nothing about how this would perform on real patients. Establishing that would require a labelled retrospective cohort of real consultations with recorded outcomes, which this project does not have and should not pretend to.

The API says so in every response, and the user interface prints it next to the score.

Intended use

What it does. Given the structured encounter a physician has recorded so far, it answers one question: *does this case warrant a second look before the consultation ends?*

What it outputs. A calibrated probability, a band, the deterministic criteria that fired, and the features that drove the score with their weights.

What it must not be read as. Not a diagnosis. Not a triage category. Not a probability of any disease. Not a reason to discharge anyone — a low score is the absence of a prompt, not the presence of reassurance.

Out of scope. Patients under 13 are not scored at all: every vital-sign threshold in the feature set is an adult range, and applying adult cut-points to a child produces confident nonsense. The API returns an explicit refusal with the reason rather than a number.

Design decisions

Logistic regression, not a gradient-boosted tree

A gradient-boosted tree was trained alongside as a reference and scored marginally better (ROC-AUC 0.909 vs 0.902; PR-AUC 0.831 vs 0.795). It is not deployed.

The requirement here is that a physician can read *why* a case was raised, during a consultation, and disagree with it. A linear model in named binary clinical features gives that directly: the contribution of “SpO₂ below 92” is one number, it is the same number in every case, and it can be checked against clinical expectation. That property is worth more than two points of AUC on synthetic data.

The reference model's score is reported in `eval/results/risk_model.json` so the choice is evidenced rather than asserted.

Named binary features, not embeddings

Every feature is a human-readable clinical fact. Text embeddings would very likely score better and would be unusable: “the vector was close to the escalation cluster” is not something a clinician can act on or dispute.

The threshold features also make the model **auditable**. The learned coefficients can be read against clinical expectation, and they are:

Feature	Coefficient
Focal neurological deficit	+4.63
Bleeding	+4.00
Altered mental state	+3.34
Thirst / polyuria	+2.88
Chronic cough	+2.71
Severe or sudden headache	+2.64
Respiratory rate ≥ 22	+2.49
Chest pain documented	+2.38
SpO ₂ below 95%	+2.03

That ordering is what a clinician would expect. If it were not, the model would be wrong and it would be obvious that it was wrong — which is the entire point of choosing features you can name.

Calibration

`class_weight="balanced"` deliberately distorts the intercept to trade precision for recall, so the raw output is not a probability. Because the number is shown to a clinician next to a threshold, it has to mean what it says, so the model is passed through an isotonic calibrator fitted on a held-out slice that never touched the coefficients.

	Brier score	Expected calibration error
Before calibration	0.076	0.100
After calibration	0.063	0.010

Exported as JSON, served in pure Python

Training uses scikit-learn; inference does not. The artefact is an intercept, a coefficient per named feature, and the isotonic calibrator as a step function. Three consequences:

- The served API has no scikit-learn, numpy or scipy dependency.
- The model file is `json.load`, not `pickle.load` — loading a pickle is arbitrary code execution, which is not a property a clinical system should ship with.

- Explanations are free: in a linear model the contribution of a feature *is* coefficient $\times$ value, so there is no separate explainer that could disagree with the model.

The training script **fails the build** if the pure-Python path diverges from scikit-learn by more than 1e-4 on the whole test set. It has already caught one real bug: `CalibratedClassifierCV` fits its calibrator on `decision_function()` rather than on `predict_proba()`, and feeding it the wrong quantity produced plausible-looking probabilities that did not match the evaluated model at all.

Current parity: $\max |\Delta| = 4.8\text{e-}05$ across 1,500 test rows.

Data

Source	Synthetic — <code>backend/app/ml/synthetic.py</code> , seed 20260911
Cohort size	6,000 encounters
Split	3,375 fit / 1,125 calibration / 1,500 test
Positive rate (test)	20.1%
Real patient data	None

The generative process

Records are built in three layers, and the third is what makes the evaluation informative rather than circular:

1. A **latent archetype** is drawn (URTI, ankle sprain, ACS, stroke, sepsis, appendicitis, DKA, TB...). The archetype is never a feature; the model sees only its observable manifestations, exactly as a clinician sees only the presentation.
2. **Observations** are sampled conditional on the archetype — complaint wording including the Hindi-English forms this system's transcripts contain, vitals from archetype-specific distributions, and realistic missingness (roughly a third of encounters record no vitals at all).
3. The **label** comes from a rule that is deliberately *not* a linear function of the features: a threshold on the *count* of deranged vital systems, an age $\times$ severity interaction, a comorbidity multiplier that only bites for cardiorespiratory presentations, and 4% symmetric label noise standing in for real inter-clinician disagreement.

Because the truth is non-linear and noisy, a logistic regression cannot recover it exactly. If the labels were generated linearly, the reported metrics would be measuring nothing.

Performance (held-out synthetic test set, n = 1,500)

Ranking

Metric	Value
ROC-AUC	0.902
PR-AUC (average precision)	0.795
Baseline PR-AUC (positive rate)	0.201

At the operating threshold (0.11)

Metric	Value
Precision	0.664
Recall (sensitivity)	0.821
Specificity	0.896
F1	0.734
False-positive rate	0.104
False-negative rate	0.179

Confusion matrix: TN 1074, FP 125, FN 54, TP 247.

Accuracy is deliberately not reported as a headline. Predicting “never escalate” would score 80% on this cohort. For an imbalanced problem where the two errors have wildly different costs, accuracy is the metric that hides the failure.

How the threshold was chosen

Not left at 0.5. It is the lowest threshold whose precision still clears 0.60, because the two errors are not symmetric in either direction:

- A missed serious presentation is the failure that hurts a patient.
- A prompt that fires too often is *also* a patient-safety failure, one step removed: clinicians learn to dismiss the panel, and then dismiss the one that mattered. Alarm fatigue is not a UX complaint.

0.60 precision is the point at which two out of three prompts are worth acting on, which is defensible for a “look again” nudge and would not be for anything that changed a patient’s pathway automatically.

Deterministic criteria (not learned)

The model is only half the escalation path. `backend/app/clinical/risk.py` checks a set of published cut-points and classic red-flag combinations independently, and **a deterministic criterion is never overridden by a low model score**:

- SpO₂ below 92%, systolic BP below 90, respiratory rate ≥ 25
- Focal neurological deficit, altered mental state, syncope

- Chest pain with breathlessness; chest pain with known cardiac disease
- Severe headache with neck stiffness; fever with neck stiffness
- Bleeding while on an anticoagulant
- Abdominal pain with fever; vomiting with rapid breathing; thirst and polyuria with vomiting; chronic cough with weight loss; weight loss with night sweats; haemoptysis

A learned model — especially one trained on synthetic data — must not be able to talk the system out of a hard safety rule. Losing the model artefact entirely degrades to these criteria rather than to nothing.

End-to-end escalation evaluation

Measured on the synthetic red-flag set (`backend/eval/red_flag_cases.json`), covering the model and the deterministic criteria together:

Sensitivity on 12 red-flag presentations	12/12 (100%)
False-alarm rate on 12 benign controls	1/12 (8%)

The benign set deliberately contains **near-misses** for each hard criterion — abdominal pain without fever, chronic cough without weight loss, vomiting without rapid breathing — because a false-alarm rate measured only on obviously benign cases is not evidence of anything.

The single false alarm is a 48-year-old smoker with a three-week cough, flagged by the model at $p=0.42$ rather than by a hard criterion. It is left in the results rather than tuned away: that patient arguably *is* worth a second look, and moving the threshold to make the number prettier would be fitting to the evaluation set.

Known limitations

1. **No clinical validation.** Stated first because it is the most important.
 2. **Adults only.** Under 13 is refused rather than approximated.
 3. **The feature lexicon is finite.** A presentation phrased in words not in `SYMPTOM_PATTERNS` will not be recognised. The red-flag evaluation found exactly this: appendicitis, DKA and TB were missed because the vocabulary to describe them was absent, not because the model was miscalibrated. The lexicon was extended and the model retrained; the same gap will exist for presentations nobody has thought of yet.
 4. **Negation is heuristic.** “No chest pain” is handled by a 30-character look-back window. It will fail on unusual constructions.
 5. **English and Hinglish only.** Other Indian languages are not covered.
 6. **No fairness evaluation.** The synthetic cohort has no caste, income, religion or region attributes, so there is nothing to measure subgroup performance across. This is a real gap, not an oversight: doing it properly needs real data with those attributes and a considered decision about collecting them.
 7. **Documentation quality is a feature.** `information_sparse` means a poorly documented encounter scores differently from a well documented one. That is intentional — absent information is genuinely a reason to look again — but it does mean the score partly reflects the note rather than the patient.
-

Maintenance

- `make train` rebuilds the model, the metrics and this card's numbers from the seeded cohort. The build is deterministic: CI retrains and fails if the committed artefact does not reproduce exactly.
- Adding or removing a feature invalidates the artefact. `risk_model.load()` refuses to serve a model whose `feature_order` no longer matches the code, because applying the wrong coefficient to the wrong feature would still produce a plausible-looking number.
- Every gate in this card runs in CI on every push.